A UVM-Based Per-Instruction Verification Methodology for the Vortex RISC-V GPGPU

Samuel Moussa, Steven Ibrahim, Ahmad Sudky, Ahmad Fawzy, Abanoub Nabil
Department of Electronics and Communications Engineering
Faculty of Engineering, Minia University, Egypt
Supervised by Dr. Hossam Hassan and Assoc. Prof. Dr. Alhassan SAYED

***Abstract*—General-purpose GPUs invert the assumptions of mainstream processor verification: the device under test is a program-executing bus *master* whose stimulus is compiled kernels rather than bus transactions, and whose SIMT execution model—warps, thread masks, divergence, barriers—lies outside the domain of standard RISC-V reference models and verification interfaces. We present a complete, fully configurable UVM verification environment for Vortex, an open-source RISC-V GPGPU, whose checking depth reaches RVVI-style *per-instruction lockstep* against a stepping functional reference model (SimX) integrated over DPI-C—to our knowledge the first such flow for a SIMT GPGPU. The methodology contributes five SIMT-specific stream-alignment rules that no scalar-CPU lockstep encounters; a sound, region-filtered per-lane load-data compare; and a two-pass load-value feed that renders racy, fenceless multi-core programs instruction-granularity verifiable (residual 0 over 5,432 retirements on the flagship case), with a precisely characterized soundness boundary at asynchronous interrupt timing. Every verdict class is proven non-vacuous by permanent fault-injection tests. The environment achieves 98.1% covergroup-bin and 94.7% total coverage on the primary configuration (50/50 runs) and 94.6% at two clusters, using only machine-generated, RTL-cited structural exclusions. Beyond the methodology, we report the defects and limitations the flow exposed in *both* models: RTL findings including a JALR target-LSB ISA deviation whose skewed PC propagates into architectural results, a non-scaling watchdog macro (`1**N`), silent misaligned-access corruption, and a reset relay that drives an *unknown* reset for one cycle into every module behind it—the last found by restoring an assertion that had been guarded off during bring-up, and whose repair *reduced* branch coverage because part of that coverage had been obtained from the unknown-reset state itself; and reference-model findings including a misaligned-fetch bug found by the lockstep itself. We close with a prioritized enhancement roadmap for both the RTL and the reference model.**

***Index Terms*—GPGPU verification, UVM, RISC-V, SIMT, lockstep, RVVI, functional coverage, fault injection, reference models, Vortex**

I. Introduction

Open-source RISC-V CPU cores enjoy a mature dynamic-verification ecosystem: constrained-random instruction generators (riscv-dv [5]), portable reference simulators (Spike [6]), and per-instruction comparison flows in which every retirement is checked against a stepping golden model through a standard interface (RVVI [4], as industrialized by OpenHW core-v-verif [3] and ImperasDV). None of this transfers directly to a GPGPU. Vortex [1], [2] executes RV32IMAF extended with six SIMT instructions (`wspawn`/`tmc`/`split`/`join`/`bar`/`tex`); Spike cannot execute them, no RVVI transactor models per-lane thread masks, and the DUT is an AXI *master* that fetches its own stimulus from memory—there is no sequence-item stimulus path to randomize.

This paper presents a UVM environment and methodology that closes that gap, and reports what the resulting checking depth revealed about both the RTL and its reference model. Our contributions:

1) **A complete, configurable UVM environment for a SIMT GPGPU** (Section III): role-inverted agents (DUT = AXI master, agents = reactive slaves), a DPI-C–integrated functional golden model rebuilt per configuration, a passive RVVI-style observability layer, and a bidirectional end-state scoreboard—any *clusters* $\times$ *cores* $\times$ *warps* $\times$ *threads* topology from runtime options (plusargs) with elaboration-time parameter self-checks.
2) **Per-instruction lockstep for SIMT** (Section V): five alignment rules—uuid-keyed program-order sorting, mask-union record aggregation, per-lane compare scoping, and volatile-CSR exclusion—that make CPU-style lockstep sound for a SIMT machine, validated lane-exact across a $2 \times 2 \times 3 \times 2$ configuration matrix.
3) **A sound two-pass load-value feed** (Section VI) that makes racy, fenceless multi-core programs—architecturally undefined and previously unverifiable—instruction-granularity verifiable, with the *residual as the verdict* and non-vacuity proven by injection; plus a characterized soundness boundary at asynchronous interrupt timing (Section XI).
4) **An evidence-classified findings register** (Sections IX–X): RTL bugs, hazards, and observability limits, and reference-model defects—each with `file:line` evidence and a disposition—including a reference-model fetch bug found *by* the lockstep, demonstrating that a lockstep flow verifies the golden model as much as the DUT.
5) **Honest coverage closure** (Section VIII): 98.1%/94.7% (covergroup bins/total) on the primary configuration

using only machine-generated, per-configuration, RTL-cited structural exclusions, with unreachable-versus-unhit distinguished by evidence and structural ceilings root-caused rather than waived.

Section XII distills the findings into a prioritized enhancement roadmap for the RTL, the reference model, and the methodology itself.

II. Background and Related Work

A. Vortex

Vortex [1], [2] is an open-source RISC-V GPGPU organized as clusters → sockets → cores → warps → threads, with per-core instruction/data caches, optional shared L2/L3, an immediate-post-dominator (IPDOM) reconvergence stack for divergence, and hardware barriers. Execution units comprise ALU, FPU (FPnew), LSU, SFU (CSR/warp control), and a tensor unit (TCU). The verified build is RV32IMAF at RTL pin `7a52ee5`, primary configuration 1 cluster / 1 core / 4 warps / 4 threads ("1CL/1C/4W/4T") with an AXI memory interface, simulated on QuestaSim 2021.2. Notably, Vortex has *no trap architecture*: there are no illegal-instruction, misaligned-address, or CSR exceptions—a property with verification consequences developed in Section IX.

B. Reference models

Vortex ships SimX, a functional (non-timing) simulator maintained with the RTL. SimX's emulator steps per instruction, returns a retirement trace with full SIMT architectural state, and is decoupled from its timing model—structurally an RVVI-shaped golden model. Spike cannot execute the six SIMT instructions, so SimX is the primary golden model and Spike serves as a *secondary, independent* base-ISA cross-check. That audit has been carried out: on the same linked ELF, Spike, SimX and the DUT retire exactly 11,076 architectural writebacks and agree on every program counter, destination register and value—zero mismatches—with non-vacuity proven by injecting a fault at one record and confirming it is named. Its scope is bounded by construction and we do not overstate it: Spike is a scalar ISS, so the audit covers warp 0 / lane 0 base-ISA execution only and stops at the first Vortex custom operation. **The SIMT axis therefore has no independent reference**, and will not obtain one from this direction. The consequence—that the primary golden model is maintained by the same project and must itself be treated as a verification target—is examined in Section X.

C. Related verification flows

core-v-verif [3] established step-and-compare verification for CPU cores against ImperasDV via RVVI [4]; riscv-dv [5] supplies constrained-random programs. Our environment descends from this methodology family and extends it in three directions: to a bus-master DUT (role-inverted agents), to SIMT semantics (per-lane masked compare, divergence, multi-record retirements), and to weakly-ordered multi-core programs (the two-pass load feed). We are not aware of a prior per-instruction lockstep flow for a SIMT GPGPU.

III. Verification Environment

Fig. 1 shows the environment: active agents drive the launch protocol and serve memory on the left, the functional reference runs in lockstep on the right, and a passive probe layer feeds the two checkers and coverage at the bottom.

A. Role-inverted agent architecture

A GPGPU DUT executes programs; it is not driven by them. The environment therefore inverts the usual UVM roles. Five agents surround the DUT: *host* and *device-configuration-register (DCR)* agents (active) drive the launch protocol—kernel base address, argument pointers, thread-count device-configuration registers—through the platform's control interface; the *AXI* agent is an active *responder*, a reactive slave backed by a memory model that services the DUT's fetch and data traffic; a *memory* agent covers the non-AXI configuration of the same model; and a passive *status* agent observes busy/completion state, the retired-instruction count, and the program counter. A virtual sequencer coordinates launch sequences. Stimulus is therefore a *program* (a compiled ELF), and the UVM test class controls only *how* it is launched, configured, and perturbed—tests and programs are orthogonal and composed at the build level.

B. Golden-model integration

SimX is linked into the simulation via DPI-C (`simx_init`/`load`/`dcr_write`/`run` plus a per-retirement record export). Both the RTL and SimX are rebuilt per configuration so that DUT and reference always agree on topology. Golden-model crashes are trapped by a signal handler and mapped to a sentinel exit code, so a reference-model failure is *classified* (Section IV) rather than crashing the run or silently passing it.

C. Passive observability layer

All white-box visibility is bound, passive, and never a checker:

- a *commit probe* bound to the retire arbiter of every core, capturing each retirement beat $\{uuid, wid, PC, rd, wb, tmask,$ per-lane data$\}$—where *uuid* is the RTL's unique per-instruction identifier, *wid* the warp id, and *tmask* the active-thread mask—across all clusters, sockets, cores, and issue lanes;
- an *LSU writeback probe* bound to the load-store slice, capturing true per-lane load values after sign/zero extension (required because load data is not observable at the commit arbiter, Section IX);
- an RVVI-style interface and UVM monitor that publishes the merged retirement stream through an analysis port, following the core-v-verif `uvma_rvvi` pattern;
- a scheduler probe and instruction-class probes feeding functional coverage.

Lockstep capture is plusarg-gated; with the gate off, runs are proven byte-identical to the plain environment.

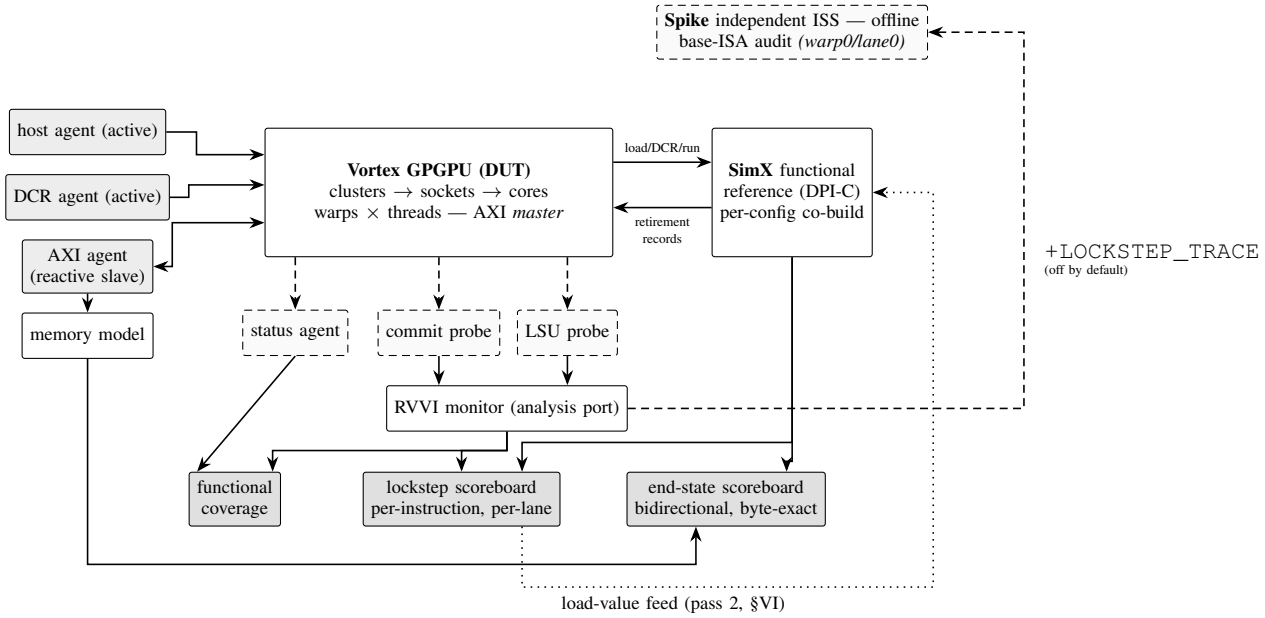

Fig. 1. Environment architecture. Solid gray boxes are active UVM agents; dashed boxes are passive probes and offline tools (never checkers in the run); dark boxes are the two verdict-rendering scoreboards and coverage. The DUT is the bus master; stimulus is a compiled program in the memory model. **Two references, with different roles:** SimX is the *primary* golden model, stepped live over DPI-C, and models SIMT; Spike is a *secondary, independent* scalar ISS used offline on an exported retirement trace to cross-check the base-ISA subset only (warp 0 / lane 0, stopping at the first custom op). SimX is co-designed with the RTL and is therefore not independent; Spike is independent but cannot execute a SIMT kernel. Neither substitutes for the other, and the SIMT axis has no independent reference (§XI).

D. Checkers

Two independent checkers render verdicts:

1) **Bidirectional end-state scoreboard.** After completion, every word the DUT wrote is compared byte-exact against SimX (forward pass, up to 32,836 words in one test), *and* every word SimX wrote that the DUT never wrote is checked (reverse pass, catching dropped stores). Sub-word stores are handled with per-byte validity masking; IEEE-rounding-legitimate FP differences are tolerance-compared; read-only/GOT sections are excluded by region.
2) **Lockstep scoreboard** (Section V): every retired instruction compared per active SIMT lane.

A third, orthogonal gate makes the run verdict *assertion-aware*: any RTL runtime assertion firing fails the run with a distinct exit code, and failing runs are excluded from coverage merging. A dedicated negative test (a deliberate misaligned load whose value is discarded) keeps this gate honest: it must fail through the assertion path alone while the scoreboards pass.

E. Configurability with self-checks

One compiled environment runs any topology from plusargs (`+CLUSTERS`/`+CORES`/`+WARPS`/`+THREADS`); interface widths (e.g., the 50-bit memory tag) are derived from the RTL package, and elaboration-time assertions compare every UVM parameter against its DUT counterpart, failing loud on mismatch. Validated topologies include 1CL/1C/4W/4T, 1CL/2C, 2CL/2C/4W/4T, 4C/2W, and 8C/8W/2T.

F. Protocol assertions

An AXI4 SVA layer ($\sim$15–18 protocol properties plus 11 handshake-stability assertions) checks burst legality, outstanding-count consistency, and signal stability under backpressure inline on the DUT's AXI interface; slave-side *throttle* (ready wait-states) and *flood* (back-to-back R beats) stress modes—plusarg-gated and proven byte-identical when off—exercise the stability properties.

IV. Verdicts and Non-Vacuity

Every run renders one of four verdicts: PASS, FAIL, UNVERIFIABLE, or (for lockstep runs) END-STATE-VERIFIED with an instruction-granularity residual. UNVERIFIABLE is first-class: a run where the golden model cannot render a verdict (e.g., a reference-model abort) is classified with root-cause evidence—never force-compared, never silently dropped, and never counted as a pass.

Verdicts are only as good as their ability to go red. The environment therefore carries *permanent negative fault-injection tests* as regression guards:

- **wrong-value injection**: one bit of one DUT store is flipped; the end-state scoreboard must catch it;
- **dropped-store injection**: one DUT store is suppressed; the reverse scoreboard pass must catch it;
- **lockstep injection**: one bit of one retirement is flipped; the lockstep comparator must catch it at the exact uuid/PC/lane;
- **assertion-gate guard**: the misaligned-load negative test must fail through the RTL-assertion path alone.

All four must stay red on injection after any checker change. This discipline, rather than any single checker, is what allows the coverage and pass-rate numbers below to be taken at face value.

V. Per-Instruction Lockstep for SIMT

A. Architecture

The DUT-side probe stream and the SimX per-retirement stream meet in a lockstep scoreboard that aligns both streams per (core, warp) and compares each retirement per active SIMT lane: PC, destination register, and written value, with a four-way outcome taxonomy (matched / field mismatch / data mismatch / orphan) and drain-empty checks on both sides so that dropped or extra retirements are caught.

B. Five SIMT-specific alignment rules

A scalar-CPU RVVI flow never meets the following; each rule was forced by simulation evidence and is, we believe, transferable to any SIMT lockstep implementation.

1) Retire order is not program order: Within one warp, the commit arbiter retires whichever execution unit is ready first (`VX_commit.sv:56-71`); an earlier-issued instruction was observed retiring after two later ones. The functional model retires in strict program order. *Rule: align by the per-warp issue counter (uuid), sorted—never by position or cycle.*

2) The golden model's uuid is not a cross-key: SimX leaves its retirement uuid at zero (Section X), so DUT and reference uuids cannot be matched directly. *Rule: the alignment key is per-(core, warp) program order; the DUT uuid's high bits—which encode* (*CORE_ID* $\ll$ *NW_BITS*) + *wid* (`VX_uuid_gen.sv:40`)*—supply the (core, warp) attribution with no RTL change.*

3) One instruction is not one retirement record: A single load can appear as several commit records sharing one uuid with partial, overlapping thread masks (observed: masks `0xd` then `0xe`), because the LSU commits lanes as their memory responses arrive—distinct from clean SIMD-beat splitting. *Rule: aggregate DUT records by uuid with thread-mask union before comparing; never aggregate by start/end-of-packet flags.*

4) Load data needs its own tap and its own soundness filter: The commit arbiter's data field is stale for loads (the LSU writes the register file on its asynchronous response path). A dedicated LSU writeback probe recovers true per-lane values—but a naive compare is *unsound*: loads of uninitialized stack or local memory legitimately differ between models (429 false mismatches on a known-good kernel). *Rule: compare a load lane only when the reference-model effective address lies in the verifiable data region and the golden value is not the initialization poison; defer everything else to the end-state check.* With this filter the load compare is exact and on by default (74 in-region lanes compared, 113 filtered, 0 false mismatches on the reference kernel).

5) Performance-counter CSRs are model-divergent by definition: `mcycle`/`minstret`/`mhpmcounter*` differ between a timing-accurate DUT and a functional model (observed off-by-one). *Rule: the golden model flags the machine-performance-monitor CSR range as volatile and the comparator excludes their data (PC and destination still checked)—the standard RVVI exclusion class.*

TABLE I
Lockstep configuration matrix (all lane-exact: 0 mismatches, 0 orphans)

Configuration	Matched writebacks
1CL/1C/4W/4T (vector-add kernel)	1,035 / 1,035
1CL/1C/4W/4T (nested divergence)	2,668 / 2,668
1CL/2C/4W/4T	1,801 / 1,801
2CL/2C/4W/4T	3,333 / 3,333
1CL/1C/2W/2T	855 / 855
1CL/1C/8W/4T	1,423 / 1,423

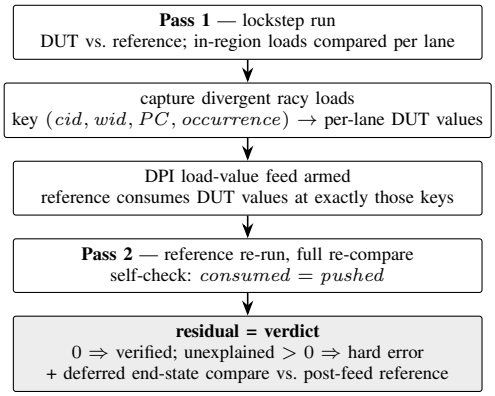

Fig. 2. The two-pass load-value feed. Only captured racy loads are fed; every other divergence in pass 2 remains an error, so the mechanism cannot mask a DUT bug.

C. Cross-configuration validation

The lockstep was validated lane-exact—zero mismatches, zero orphans—across a configuration matrix spanning *NCL* $\in \{1,2\}$, *NC* $\in \{1,2\}$, *NW* $\in \{2,4,8\}$, *NT* $\in \{2,4\}$ (Table I), including a nested-divergence kernel (asymmetric 3v1 $\rightarrow$ 2v1 $\rightarrow$ 1v1 splits) that exercised the mask-union aggregation through divergence and reconvergence with no comparator changes.

VI. Verifying Racy Programs: the Two-Pass Load-Value Feed

A. The problem

A fenceless program run on N shared-memory cores is architecturally undefined: any interleaving of the cross-core stores is legal, so a functional model stepping cores in a fixed order and a timing-accurate DUT legitimately read *different* values at racy loads—and every downstream value forks. End-state comparison can only classify such runs unverifiable. Industrial RVVI flows solve the analogous CPU problem by feeding DUT-observed load data to the reference model over a "load bus"; no such mechanism existed for a SIMT multi-core.

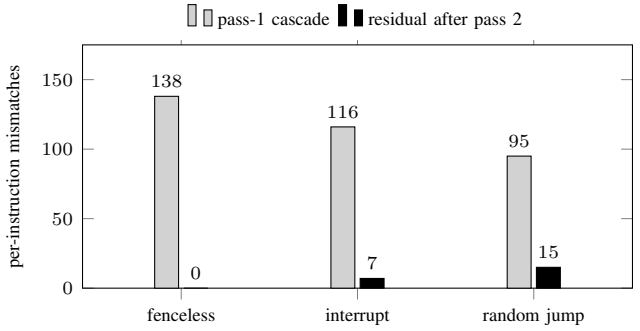

Fig. 3. Divergence collapse under the load feed at 2CL/2C/4W/4T. The fenceless case reaches residual 0 (fully verified). The nonzero residuals are the two soundness boundaries of Section XI: interrupt-delivery timing (7, keying-independent) and control-flow-steering races (15, no fixed point); both end-state compares pass.

B. The mechanism

We implement the load bus as a *sound two-pass trace replay* (Fig. 2):

1) **Pass 1** runs lockstep normally and captures every divergent in-region load, keyed by $(cid, wid, PC, occurrence)$—a key robust to instructions inserted at different points in the two streams.
2) The captured DUT per-lane load values are fed into the reference model through a DPI overlay at its single load site.
3) **Pass 2** re-runs the reference in-process, following the DUT's racy loads, and re-compares everything.

Soundness rests on three properties: (i) *the residual is the verdict*—any pass-2 divergence not explained by a fed racy load remains a hard error; (ii) a `consumed==pushed` self-check guards feed alignment; (iii) pass-1 divergences are demoted to diagnostics only when the feed is armed. The end-state comparison is deferred and re-run against the post-feed reference, so final-memory equivalence is still independently checked.

C. Flagship result

On the pinned fenceless test at 2CL/2C/4W/4T—previously undecidable—pass 1 identified 20 racy loads causing 138 cascaded mismatches; pass 2 reached **residual 0 over 5,432/5,432 retirements** (Fig. 3), and the deferred end-state compare passed. The run is thereby *positively verified* equivalent modulo the architecturally-undefined racy loads, not merely excused. Injection guards remained red, and the default (no-feed) path remained byte-identical on regression kernels.

D. Failure localization

The same lockstep replay pinpoints first divergences exactly. For the fenceless case the first divergent instruction is `mulhu s0,s3,a3` at PC `0x800004f4` (sequence 278, cluster-1 cores only; cluster-0 byte-exact), and the divergence is already present in its *inputs*: an upstream shared load at `0x80020618` where the DUT reads the pristine ELF initialization value (`0x7aea0e77`) while the reference—having interleaved another core's store first—reads `0x7a000e77`. This confirms, at instruction granularity, a reference-model memory-ordering artifact and *not* a DUT bug; end-state comparison alone could never make that distinction.

VII. Stimulus

Directed kernels ($\sim$30, all compiled with the Vortex toolchain, all byte-exact against the reference): arithmetic and memory baselines; all 13 FPU operation classes; tensor-unit WMMA including multi-warp collective launches; nested-divergence towers; barriers under partial thread masks; `wspawn`/`tmc` sweeps; cache miss-status-holding-register (MSHR) and memory-pressure stress; a 232 KB-text kernel; a 256 KB high-entropy store stress; division corner cases; CSR writes under SIMT masks; and VOTE/SHFL operations.

Constrained-random: a riscv-dv [5] pipeline targeting rv32im, with GPU-specific post-processing (machine-mode CSR stripping, `ecall`→`ebreak`) and end-state verification; twelve random-program seed profiles run in the suite. Two riscv-dv profiles are excluded as *unimplementable* on this DUT—unaligned-load/store (the DUT has no misaligned support, Section IX) and illegal-instruction (no trap architecture)—and jump-stream generation is sanitized to keep the generator's deliberate JALR-LSB probing away from an RTL deviation it would otherwise trip (finding R1, Section IX); the deviation itself is reported, not hidden.

Protocol stress: the AXI throttle and flood modes of Section III, plus host/DCR launch-space sweeps.

VIII. Coverage Methodology and Results

A. Closure discipline

Three rules govern closure. (1) *Exclusions are structural, RTL-cited, and machine-generated*: a per-configuration generator emits every waiver with a `file:line` citation (e.g., a write-response stability assertion is unreachable because the adapter hardwires `m_axi_bready = 1'b1`, `VX_axi_adapter.sv:313`), and the merge flow verifies zero ineffective waivers. Exclusions are keyed to the configuration—e.g., the global-barrier path is excluded only at single-core, where it is structurally unreachable, and kept at ≥ 2 cores. (2) *Reachable-but-unhit is left red*: bins that stimulus could reach but did not are reported uncovered, never waived. (3) *Ceilings are root-caused*: the toggle plateau ($\sim$78%) was traced to the write-through cache configuration (512-bit line write-data fields never driven) and constant high PC/address bits for realistic programs; an adversarial maximum-entropy kernel moved aggregate toggle by $+0.02\%$, establishing the ceiling as structural. Coverage banks are reported *per configuration*: merging coverage databases across topologies was shown to be invalid (instance-set inflation deflates by-instance percentages).

TABLE II
COVERAGE BANKS (QUESTASIM; PER-CONFIGURATION, NEVER BLENDED)

Metric	1CL/1C/4W/4T	2CL/2C/4W/4T
Covergroup bins (raw)	370/377 = 98.1%	989/1032 = 95.8%
Covergroup (weighted)	99.8%	99.5%
Statement	98.1%	98.3%
Branch	94.5%	95.7%
Condition	90.4%	88.8%
Toggle	83.3%	80.5%
Assertion	96.9%	98.9%
Directive	100.0%	100.0%
Total	**94.7%**	**94.6%**
Runs passing	51/51	50/50
Coverage instances	2 256	8 275

All runs pass at both configurations; earlier reported failures were a methodology defect (Sec. XI), not DUT divergence. *Runs*, not distinct programs: two entries re-run one program under a different bus mode. A third configuration enabling the optional shared L2/L3 caches was also banked (51 runs, all passing, 93.2% total); those levels are bypassed in the two banks above. The 1CL bank is measured on the reset-corrected design; the 2CL and L2/L3 banks predate that correction and are therefore not comparable to it on *branch* coverage.

B. Functional model

Seventeen covergroups span instruction classes per execution unit (ALU/FPU/LSU/SFU/TCU, operation-decoded), SIMT divergence crossed with IPDOM depth, warp/thread-mask crosses, barrier/`wspawn`/`tmc` behavior, a stall taxonomy crossed with IPC buckets, AXI fields, DCR and host launch spaces, and system state—each with a written sufficiency rationale.

C. Results

Table II summarizes the two banked configurations.

D. Provenance of the design under verification

The design is an open-source register-transfer model pinned at a specific revision, *with eighteen locally modified files in the synthesizable register-transfer tree* (support and co-simulation glue outside that tree is additionally modified, and the environment itself is new code). Most changes are simulator-compatibility fixes made during bring-up. We disclose this because a verification result is a statement about a specific artifact, and "upstream, unmodified" would not describe what we ran.

One of those modifications turned out to matter, and its history is instructive. A library counter shipped with two assertions checking for overflow and underflow. They fired during bring-up and were guarded off—rewritten to skip whenever their inputs were unknown—so that the environment could run at all. That guard remained for months. On restoring the original assertion and root-causing why it fired, we found a genuine defect: the design distributes reset through a relay module that *registers* reset in a flip-flop which nothing resets. Its output is therefore unknown from time zero until the first clock edge, so every module behind a relay observes an unknown reset for one cycle; a reset-conditional then takes its non-reset branch and any assertion inside evaluates on unknown operands.

We replaced the relay with an asynchronous-assert, synchronous-deassert synchronizer. The counter assertions then pass with no guard at all—twelve firings become zero—and the local modification to that counter was retired in favor of the unmodified upstream file. The defect is present in the current upstream release.

Two consequences deserve emphasis. First, **the assertion had been correct**: what was silenced was a real property of the design, not a testbench nuisance. A checker that is switched off to make a bench run is a checker whose findings are lost, and the cost here was months of an unobserved unknown-value window. Second, **fixing the defect reduced a coverage number**. Branch coverage fell from 95.1% to 94.5%: during the unknown-reset cycle, modules behind a relay had been executing their normal-operation paths, and those executions were *counted as covered branches*. With reset correct they no longer occur. Part of the previously reported branch coverage had been obtained from a state that cannot legitimately arise—which no coverage metric can reveal about itself. We report the lower figure as the correct one. (Two variables changed between those measurements, so the delta is indicative rather than isolated.)

TABLE III
RTL FINDINGS (SUMMARY)

ID	Finding	Class	Disposition
R1	JALR target LSB not cleared; odd PC propagates via AUIPC	Bug (ISA)	needs RTL fix; stimulus sanitized
R2	`STALL_TIMEOUT` uses `1**N` $\equiv$ 1; watchdog never scales	Bug (latent)	fixed; *independently fixed upstream*
R3	Misaligned access: no trap; silently retargeted/torn	Hazard	expected per SW contract; gated
R4	Core self-starts from reset; DCRs have no reset value	Hazard	worked around (reset handshake)
R5	Per-warp out-of-order commit	Quirk	handled (§V-B)
R6	One load $\rightarrow$ multiple commit records, overlapping masks	Quirk	handled (§V-B)
R7	uuid encodes flat core id + warp id	Quirk	exploited (§V-B)
R8	Load data not observable at commit arbiter	Observability	closed (LSU probe)
R9	Write path fire-and-forget (`bready` tied high)	Characteristic	cited exclusion
R10	Reset relay registers reset in a flop nothing resets; `reset_o` is X for one cycle	Bug (X source)	fixed; still upstream

IX. FINDINGS I: RTL OBSERVATIONS

Every finding is maintained in an evidence-cited register shipped with the work (one entry per finding: what was seen, `file:line` and/or run evidence, classification, disposition). Table III summarizes the RTL findings as R1–R10; the substantive entries follow.

A. An unknown reset for one cycle, found by restoring a silenced assertion (R10)

This finding is reported first because of *how* it was found rather than what it is.

The design distributes reset through a relay module. That module registers the incoming reset in a flip-flop which has no initial value and which nothing resets:

```
`PRESERVE_NET reg [R-1:0] reset_r;   // no initial value
always @(posedge clk) begin
    reset_r[i] <= reset;             // nothing resets THIS
          flop
end
assign reset_o[i] = reset_r[i / F];
```

Its output is therefore unknown from time zero until the first clock edge, so every module instantiated behind a relay observes an *unknown reset for one cycle*. A reset-conditional in such a module takes its non-reset branch—`if (X)` is not true—so logic intended to be held in reset executes, and any assertion inside that branch evaluates on unknown operands.

The path to it is the point. A library counter in the same design ships with assertions checking for overflow and underflow. Those assertions fired during bring-up, at 5, 15 and 25 ns, on the instruction- and data-cache miss-status counters. They were guarded off—rewritten to skip whenever their inputs were unknown—so that the environment could run, and the guard stayed for months. Restoring the original assertion and asking why it fired produced the diagnosis above. **The assertion had been correct the entire time**: it was reporting a real property of the design, and what had been suppressed was the report, not the problem.

We replaced the relay with an asynchronous-assert, synchronous-deassert synchronizer, so that the relay output asserts whenever reset asserts regardless of flip-flop state. The counter assertions then pass with no guard at all—twelve firings become zero—and the local modification to the counter was retired in favour of the unmodified upstream file. A fifty-one-run regression passes with the assertions armed, and both fault-injection guards were re-confirmed to fail on injection. The defect is present in the current upstream release.

Two observations generalize beyond this design. First, **a checker disabled to make a bench run is a checker whose findings are lost**, and the loss is silent and open-ended: here it was months, and the affected surface was every module behind a relay, not merely the one that happened to carry an assertion. Second, **fixing the defect reduced a coverage number** (Section VIII-D): branch coverage fell because modules behind a relay had been executing their normal-operation paths during the unknown window, and those executions were counted as covered branches. No coverage metric can report that some of its own coverage came from a state that cannot legitimately arise.

B. JALR does not clear the target LSB (R1)

The RISC-V specification requires JALR to clear the least-significant bit of the computed target. Vortex omits the clear (`VX_alu_int.sv:222`: the branch destination is the raw `rs1+imm`), and—having no trap architecture—cannot raise the instruction-address-misaligned exception the specification prescribes for a misaligned target on a non-compressed core. In the debug build (`PC_BITS = XLEN`, identity PC conversion) the odd bit survives as the *architectural* PC. Fetch silently word-aligns (`VX_fetch.sv:101`), so execution continues on correct instruction words—but the skewed PC *does* reach architectural results: every `auipc`/`la` computes $rd = PC + imm$ and inherits the skew, link-register writes accumulate it across chained jumps (observed $+1 \rightarrow +3$), and downstream loads/stores go misaligned, cascading into R3. The trigger is spec-legal stimulus: riscv-dv deliberately sets the JALR base LSB expecting the architectural clear, so roughly half of generated jumps derail—in a 12-profile suite, every profile fired misaligned-access assertions (30–7,616 per run). In the release build (`PC_BITS = XLEN-2`) a $+1$ target is masked away by representation—spec-correct *by accident*—while a $+2$ target would silently word-align where the specification demands a trap. The fix is a one-line `& ~1` at the destination adder; because the reference model deliberately mirrors the no-clear behavior, the fix must un-mirror both models together.

C. Non-scaling stall watchdog (R2)

`VX_config.vh:246` defines `STALL_TIMEOUT = 100000 * (1 ** (L2_ENABLED + L3_ENABLED))`. The intent is to scale the pipeline stall watchdog with cache-hierarchy depth, but $1^N \equiv 1$, so the threshold is constant regardless of L2/L3. On deep-hierarchy configurations the watchdog can fire spuriously on legitimately long stalls. The fix is one character (`10 ** (...)`).

D. Misaligned access: silent corruption, simulation-only detection (R3)

Misaligned data access is documented-unsupported, but the failure mode is hazardous. The byte-enable logic truncates address low bits per access size (`VX_lsu_slice.sv:159-184`): a halfword access at an odd address is silently *retargeted* to the aligned slot, and an RV32 misaligned word access reads/writes only the containing aligned word—never crossing into the next word as byte-span semantics would require—while the store-data shifter uses the full alignment offset, so enable-set and data-shift disagree: *torn bytes at a wrong address, with no error indication to software*. The only detection is a simulation-only runtime assertion (`VX_lsu_slice.sv:189`), compiled out under synthesis—silicon has zero detection. The same class recurs for CSRs: an invalid CSR access asserts in simulation (`VX_csr_data.sv:150`) instead of raising an exception. Our handling is defense in depth (the assertion-aware verdict gate fails any run that trips these) plus an upstream enhancement recommendation (Section XII). The reference model, by contrast, performs the access byte-accurately at the exact address, so any boundary-crossing misaligned access is a guaranteed DUT/reference divergence—detectable per-instruction under lockstep.

E. Startup protocol hazard

The core self-starts from reset (`VX_schedule.sv:230`) while the base device-configuration registers have no reset

value (`VX_dcr_data.sv:27`): a core that leaves reset before the host completes DCR programming fetches from an undefined base. The environment holds reset until a DCR-bootstrap-done handshake; an integration would need the same discipline or an RTL interlock.

F. Microarchitectural quirks and observability limits

Three quirks required—and validated—the alignment rules of Section V-B: per-warp out-of-order retirement (R5), multi-record partial-mask load retirement (R6), and the uuid encoding of flat core id and warp id (R7), which multi-core attribution exploits with no RTL change. One observability limit is structural: load writeback data never reaches the commit arbiter tap (R8); the dedicated LSU probe (with the soundness filter of Section V-B) closed it. Additionally, the LSU result bus broadcasts the active lane's value across all lane positions per beat (it is not a per-lane vector), and the AXI adapter hardwires write-response ready—a fire-and-forget write path that makes one class of response-stability assertions structurally untestable (cited as a coverage exclusion, and a candidate RTL robustness improvement).

X. Findings II: Reference-Model Observations

A central lesson of this work: *a lockstep flow verifies the reference model as much as the DUT*. SimX is maintained by the same project as the RTL, so shared assumptions can hide shared bugs; the per-instruction compare surfaced defects in the model itself.

A. Misaligned instruction fetch (found by lockstep; fixed)

SimX fetched at the exact byte PC (`emulator.cpp:145`). When a program's architectural PC is odd (the R1 mechanism—which the model correctly mirrors), the DUT fetches the word-aligned instruction and runs on, but SimX read misaligned bytes straddling two instructions, produced an undecodable word, and aborted—silently misclassifying every such run as unverifiable. The fix word-aligns the emulator fetch (`PC & ~3`) while leaving the architectural PC odd to match the DUT. Notably, the "obvious" fix—masking the JALR target in the model—was tried and *reverted with evidence*: it desynchronized the models (21,968 phantom PC mismatches), because it fixed the model to the specification rather than to the DUT under test.

B. Abort-on-unknown as an unverifiability source

The SimX decoder and executor abort on anything unrecognized: 56 `std::abort()` sites (35 decode, 21 execute). Aborting is arguably correct for a golden model—refusing beats fabricating—but a bare abort converts a run into an unexplained UNVERIFIABLE. Root-causing one such abort showed the DUT and model *differ by design* here: a computed wild jump landed on non-instruction bytes; the RTL decoder degrades gracefully and runs to completion while the model dies. Two mitigations are in place: decode aborts now print the faulting PC and instruction word (self-documenting), and lockstep upgrades the verdict from "nothing known" to "DUT $\equiv$ reference byte-exact for all 17,664 retirements the model executed before its crash"—corroboration end-state checking cannot provide.

C. Unpopulated retirement uuid

The model's retirement records carry $uuid = 0$, so DUT and reference uuids cannot be cross-keyed; alignment must derive from per-(core, warp) program order (Section V-B). Populating it would upgrade an alignment heuristic into a strict 1:1 assertion.

D. Timing-class divergences

Two divergence classes are definitional for a functional model against a timing-accurate DUT, and must be engineered around rather than fixed: performance-counter CSR reads (flagged volatile and excluded, the standard RVVI class), and the ordering of cross-core memory operations in fenceless programs (the model steps cores in a fixed interleaving)—the class the two-pass load feed (Section VI) renders verifiable. Asynchronous interrupt delivery is the boundary of that technique (Section XI).

E. Robustness gaps

Additional model defects surfaced during integration and are reported upstream: internal object sizing bound to compile-time defaults rather than the configured warp count (out-of-range crash at small configurations), aborts on machine-mode CSR ranges and compressed instructions that the generator legitimately emits, and a tensor-unit capability flag missing from the co-simulation build (silently classifying all tensor tests unverifiable). Each was root-caused and fixed or worked around; the pattern—*the reference model needs its own verification and robustness budget*—is the finding.

XI. Limitations and the Soundness Boundary

A. Asynchronous interrupt timing

The interrupt-random test at two clusters collapses from 116 cascaded divergences to 7 under the load feed—and does not reach zero. The residual is proven *keying-independent*: re-keying the feed from a per-warp ordinal to $(cid, wid, PC, occurrence)$ leaves it identical, disproving feed-alignment artifacts. The cause is genuine: the timing-accurate DUT and the functional model take asynchronous interrupts at different instruction boundaries, so an interrupt-affected path executes a different number of times; no load-data feeding can align *when* an interrupt fires. The end-state compare against the post-feed model passes, so the disposition is: end-state VERIFIED, instruction-granularity residual classified—not forced green. This defines the method's soundness boundary: *two-pass trace replay is a fixed point for data-only divergence; asynchronous-input timing requires a step-follower reference*.

B. Control-flow-steering races

A related boundary: when racy loaded bytes steer control flow (a random jump test), pass-2 replay walks a different path and meets *fresh* racy loads the pass-1 trace never keyed (residual 15, all load-class; the post-feed end-state compare is clean). Two-pass replay has no fixed point when races feed branches; an iterated (bounded fixed-point) feed or a step-follower is required. The verdict is left honestly red.

C. Other limitations

- **Reference-model independence.** SimX is not independent of the DUT. A base-ISA audit against Spike has since been completed—DUT, SimX and Spike retire an identical 11,076 writebacks and agree on every PC, destination and value, with non-vacuity proven by injection—but Spike cannot execute the SIMT extensions, so that independence covers the scalar subset only (warp 0 / lane 0). *The SIMT axis has no independent reference, and this is a structural ceiling rather than an unfinished task.*
- **Unverifiable residue.** A minority of random-program runs remain unverifiable due to reference-model aborts; the abort-hardening sweep (Section XII) targets zero.
- **Structural coverage ceilings.** Toggle coverage plateaus in the low 80s for root-caused structural reasons—dominated by a *read-only instruction cache* whose write-data path is elaborated but undrivable (26% of the whole gap from one subtree), plus write-through data caching and constant address high bits. We report the true value rather than gaming it, and we corrected our own attribution when measurement contradicted it (Section VIII).
- **No trap architecture.** Exception-path verification is unimplementable on this DUT (there are no exceptions); the corresponding generator profiles are excluded as unimplementable rather than skipped silently.
- **Verified build.** Findings are stated against the debug build (`PC_BITS = XLEN`) at one RTL pin; the release build changes the visibility (not the presence) of finding R1.

XII. Proposed Enhancements

A. RTL (upstream recommendations)

1) **JALR LSB clear** (`& ~1` at the destination adder), coordinated with un-mirroring the reference model—restores specification conformance and un-derails spec-legal generators.
2) **Watchdog scaling fix**: `1 ** N` → `10 ** N` in `STALL_TIMEOUT`.
3) **Misaligned-access detection in silicon**: a misaligned-address trap, or minimally a sticky error CSR—today the only guard is a simulation-only assertion and the silicon failure mode is silent data corruption.
4) **Write-response handling**: the fire-and-forget AXI write path (response-ready tied high) forfeits error observability; consuming write responses would also make the untestable stability assertions meaningful.
5) **Startup interlock**: hold cores in reset until DCR programming completes, removing the integration hazard.

B. Reference model

1) **Abort hardening** (*completed since the first draft; retained here because the classification is the transferable result*). Auditing the abort sites showed they were not one population but two. The majority lay in the disassembly formatter—off the execution path entirely, reached only when a trace line is printed—so aborting there destroyed an otherwise fully checked run because the model could not *name* an instruction; these now emit a placeholder. The remainder are genuine semantic refusals, and for those aborting is *correct*: a golden model that guessed a writeback would corrupt every later comparison while still reporting agreement. Those sites now record the PC, instruction word and offending sub-field before aborting, and the co-simulation layer returns a distinct *golden-halt* sentinel (refused at a known point) rather than the former anonymous crash code—so the instructions retired *before* the refusal remain valid evidence, and the truncated tail is excluded from the verdict rather than counted as divergence. Re-measuring the constrained-random suite afterwards showed the unverifiable class was already empty: every retained profile produced real end-state comparisons. Because nothing in the suite then exercises the halt path, it is proven non-vacuous by a gated injection, in the same discipline as the negative tests of Section IV.
2) **Populate the retirement uuid**, upgrading lockstep alignment to a strict 1:1 key.
3) **Spike base-ISA audit** (*completed*): the model's RV32IM semantics were cross-checked against an independent simulator on the non-SIMT subset, with exact three-way agreement; extending independent cross-checking *above* the scalar subset remains open and is structurally hard (§XI).

C. Upstream contribution

We regard the findings of Section IX as belonging to the Vortex project rather than to this report, and they are being submitted upstream as individual issues against the public repository, each with the exact source location, a reproducer, and our assessment of severity.[1] Two points of discipline govern that submission and are worth stating, because they are what separate a useful report from a noisy one.

First, *severity is claimed conservatively*. Of the findings, only the JALR specification deviation is a conformance bug with silent data-corruption consequences; the stall-watchdog items are verification-guard defects that make correct runs appear broken but do not affect the datapath, and we label them as such. Misclassifying a guard defect as a design defect would be the fastest way to lose a maintainer's attention.

[1]Prepared as a standing document in our repository so that the submitted text and the text analysed here cannot drift apart.

Second, *every claim is re-verified against upstream before filing*. All findings were made at a pinned revision; a finding already fixed upstream costs reviewer time and costs the reporter credibility, so re-checking each cited line against current `master` is a precondition of submission rather than a courtesy.

We also offer the independent base-ISA cross-check of Section XI back to the project. We deliberately do *not* propose extending the scalar reference simulator to the SIMT model: the warp, thread-mask and reconvergence-stack state that Vortex requires cannot be hosted by a single-hart processor model without re-architecting it, and the result would be a second implementation of the existing reference rather than an independent check on it. Scoping the independent reference to the scalar subset—which carries the overwhelming majority of dynamic instructions, and where the external simulator is the actual specification authority—is what makes the audit meaningful.

D. Methodology

1) **Step-follower lockstep** (the principal enhancement): a single-pass flow that steps the reference one retirement at a time behind the DUT with interrupt-delivery alignment—the ImperasDV pattern—which dissolves both soundness boundaries of Section XI. The RVVI monitor/analysis-port architecture already in place is its prerequisite.
2) **Bounded iterated load feed** as a cheaper partial alternative for control-flow-steering races (may not converge for interrupts).
3) **LSU address/data compare** extension of the lockstep (store addresses and data at retirement).
4) **Formal property verification** on the arbitration-heavy blocks the dynamic flow exercises least (cache MSHR, commit arbiter).
5) **Register-abstraction layer** for the DCR space; continuous integration with seeded regression, graded sign-off, and per-seed triage.

E. Toward tape-out: a categorized roadmap

The work reported here is *front-end functional verification* of an RTL model. Silicon sign-off requires substantially more, and the two categories below are deliberately separated because they demand different tooling, different skills, and—critically—different claims. Neither list is a statement of partial progress: every item is currently absent unless noted.

1) Remaining front-end (RTL functional) work:

1) **Stimulus diversity, not volume.** The generator is seed-controlled, and a ten-seed sweep across all nine profiles has since been run: ninety additional distinct programs (verified distinct by content hash), all passing, and *no measurable coverage gain*—every category bit-identical except toggle at $+0.06\%$. That is a robustness result, not a coverage one. The binding constraint is the generator's reach: it emits user-mode integer code with machine-mode control-register writes removed, so every seed explores the same region and coverage saturates inside it. The remaining work is stimulus of a different *kind*—privileged and exception behaviour, and bus error responses—not more samples of the same kind.
2) **Error and exception verification.** The bus responder always returns `OKAY`; no `SLVERR`/`DECERR` injection exists, so no error-recovery path is exercised. Architectural exception and interrupt stimulus is likewise absent, and must be designed around the observation that this design terminates by deasserting a busy signal rather than by a breakpoint instruction.
3) **Formal property verification** on arbitration-heavy control (cache miss-status handling, commit arbitration), where dynamic simulation is weakest and state space is small enough for proof.
4) **X-propagation and reset randomization.** Uninitialized-state bugs are invisible to a two-state functional flow; several unreset elements have already been identified structurally but never exercised under randomized reset.
5) **Configuration-matrix breadth.** Three points are sampled of a parameter space spanning cluster, core, warp, thread and cache-tier dimensions.
6) **Coverage-model provenance.** The functional model is self-authored rather than traced to a specification document, so coverage closure measures the model, not the specification.
7) **Independent SIMT reference.** The scalar base-ISA axis has an independent cross-check; the SIMT axis does not, and this is a structural ceiling rather than an unfinished task.

2) Back-end and ASIC sign-off work (entirely out of scope here):

1) **Gate-level simulation** on the synthesized netlist, then with back-annotated timing, to catch synthesis mismatches and timing-dependent behavior a register-transfer flow cannot see.
2) **Static timing analysis** sign-off across process, voltage and temperature corners.
3) **Design-for-test:** scan insertion, ATPG pattern generation and fault-coverage sign-off, plus memory built-in self-test.
4) **Clock- and reset-domain-crossing analysis**, structural and functional, including metastability modeling.
5) **Lint and structural sign-off** against a synthesis-clean rule set.
6) **Low-power verification:** power intent, retention and isolation checking, and power-aware simulation.
7) **Physical-design closure:** floorplan, place and route, parasitic extraction, signal- and power-integrity analysis.
8) **Equivalence checking** between register-transfer source, synthesized netlist and post-layout netlist.
9) **Post-silicon:** bring-up plan, characterization, and a path from silicon failures back into this regression.

A useful way to read the two lists: the front-end items

would raise the strength of the claims made in this paper, whereas the back-end items are prerequisites for a different claim entirely—that the design is manufacturable and will function in silicon. Nothing in this work speaks to the latter.

XIII. Conclusion

We presented a UVM-based methodology that brings per-instruction golden-model verification—the standard of care for RISC-V CPU cores—to a SIMT GPGPU. The environment is complete and configurable: role-inverted agents around a bus-master DUT, a per-configuration co-built functional reference over DPI-C, a passive RVVI-style observability layer, bidirectional end-state checking, and an injection-proven non-vacuity discipline behind every verdict. The lockstep layer contributes five transferable SIMT alignment rules, a sound region-filtered load compare, and a two-pass load-value feed that turned an architecturally-undefined racy multi-core program from unverifiable into positively verified (residual 0 over 5,432 retirements), while honestly delimiting its own soundness boundary at asynchronous interrupt timing. Coverage closed at 98.1% covergroup bins / 94.7% total on the primary configuration with only machine-generated, RTL-cited exclusions.

The checking depth paid for itself in findings on both sides of the comparison: in the RTL, a JALR ISA deviation with architectural consequences, a non-scaling watchdog macro, and a silent misaligned-access corruption mode with no silicon detection; in the reference model, a misaligned-fetch bug that had been silently discarding verification results, found by the lockstep itself. Both registers ship with the work, each entry evidence-cited and dispositioned. We believe the methodology—and its central lesson, that the reference model must be verified alongside the DUT—transfers directly to other open-source SIMT designs.

Acknowledgment

This work was carried out as a graduation project at the Faculty of Engineering, Minia University, under the supervision of Dr. Hossam Hassan and Assoc. Prof. Dr. Alhassan SAYED, whose guidance is gratefully acknowledged.

References

[1] B. Tine, K. P. Yalamarthy, F. Elsabbagh, and H. Kim, "Vortex: Extending the RISC-V ISA for GPGPU and 3D-graphics," in *Proc. 54th IEEE/ACM Int. Symp. Microarchitecture (MICRO-54)*, 2021, pp. 754–766.

[2] F. Elsabbagh, B. Tine, P. Roshan, E. Lyons, E. Kim, D. E. Shim, L. Zhu, S. K. Lim, and H. Kim, "Vortex: OpenCL compatible RISC-V GPGPU," in *Workshop on Computer Architecture Research with RISC-V (CARRV)*, 2019.

[3] OpenHW Group, "core-v-verif: functional verification project for the CORE-V family of RISC-V cores," [Online]. Available: https://github.com/openhwgroup/core-v-verif

[4] OpenHW Group / Imperas, "RVVI: RISC-V Verification Interface," [Online]. Available: https://github.com/riscv-verification/RVVI

[5] CHIPS Alliance, "riscv-dv: SV/UVM based instruction generator for RISC-V processor verification," [Online]. Available: https://github.com/chipsalliance/riscv-dv

[6] RISC-V International, "Spike, the RISC-V ISA simulator," [Online]. Available: https://github.com/riscv-software-src/riscv-isa-sim

[7] *IEEE Standard for Universal Verification Methodology Language Reference Manual*, IEEE Std 1800.2-2020, 2020.